



AJE 移植指南

1 术语/缩略词

SALELF [®] 2	: 以下简称为 EF2/ELF2	精灵 2 系列 FPGA
SALEAGLE [®] 4	: EF4X20/EG4A20/EG4S	猎鹰系列 FPGA
SALFPGA [®]	: Field Programmable Gate Array	现场可编程门阵列
TangDynasty [®]	: 以下简称为 TD	安路 FPGA 开发软件
AJE	: Anlogic JTAG Embedded	安路 aje 文件
JTAG	: Joint Test Action Group	联合测试工作组

2 关于本手册

本手册主要介绍 AJE 文件格式以及 AJE2JTAG 的移植，并在 SOC 平台上验证。

3 AJE2JTAG 简介

将 AJE2JTAG 系统代码嵌入到 MCU/CPU 中，可以使 CPU/MCU 很方便的使用 GPIO 通过 JTAG 接口对外部可编程器件进行配置。结合 TD 生成的 SVF 文件及其 SVF2AJE 工具产生的*.aje 文件，用户能够快速配置安路 CPLD/FPGA，以实现在线更新功能。

4 AJE 文件格式说明

AJE 文件由文件头和 SVF 数据两部分组成。

4.1 文件头结构

文件头组成部分：

CRC	版本	是否压缩	内存大小
-----	----	------	------

描述如下：

字段	字节数	描述
CRC	3	CRC 指令编码（0x61，1 字节） CRC 校验码（2 字节）
版本	8	版本信息 例如，AL__1.0
是否压缩	1	0xF1 时，表示压缩模式
内存大小	2+	MEM 指令（0x19，1 字节） 最大内存大小（所占字节数与数值大小有关）

总计：13 字节以上。

4.2 SVF 数据格式

SVF 数据组成部分:

厂商	行头+指令编码+数据	行头+指令编码+数据		文件结束标记
----	------------	------------	-------	--------

各字段说明如下:

字段	字节数	描述
厂商	2	厂商指令编码 (0x56, 1 字节) 厂商编号 (1 字节)
行头	2+	行头编码 (0x6E, 1 字节) 行长度 (指令编码+数据的长度, 1 字节及以上)
指令编码	1	SVF 命令编码
数据	1+	SVF 命令所带的数据
文件结束标记	1	文件结束编码 (0x7F)

其中, 指令及其数据格式:

- 固定数据长度的指令: 指令编码 + 数据;
- 可变数据长度的指令: 指令编码 + 数据长度 + 数据;
- 数据长度格式: 每个字节中的最高位用于表示在其之后的一个字节是否也用于记录数据长度。
当最高位为 1 时, 表示后一个字节也用于记录数据长度。

4.3 文件转换示例

以 refresh.svf 为例, 对 AJE 文件格式进行说明。其中 refresh.svf 内容如下:

```
TRST OFF;
ENDIR IDLE;
ENDDR IDLE;
STATE RESET;
STATE IDLE;
FREQUENCY 1E6 HZ;
//Loading device with 'idcode' instruction.
SIR 8 TDI (06) SMASK (ff) ;
RUNTEST 15 TCK;
SDR 32 TDI (00000000) SMASK (ffffffff) TD0 (03004c37) MASK (ffffffff) ;
RUNTEST 15 TCK;
//Loading device with 'refresh' instruction.
SIR 8 TDI (01) SMASK (ff) ;
RUNTEST 15 TCK;
SIR 8 TDI (1f) ;
RUNTEST 200000 TCK;
```

通过该 SVF 文件生成的 AJE 文件如下所示:

```

0000:0000 61 CC 1C 41 4C 5F 5F 5F 31 2E 30 F2 62 20 56 01 5E 10 01 03
0000:0014 5E 10 05 01 5E 10 06 01 5E 10 04 00 5E 10 04 01 5E 20 07 C0
0000:0028 84 3D 5E 28 0E 08 0F 60 17 5E 20 04 01 14 19 5E 90 01 0D 20
0000:003C 0F 00 00 00 00 10 EC 32 00 C0 11 FF FF FF FF 17 5E 20 04 01
0000:0050 14 19 5E 28 0E 08 0F 80 17 5E 20 04 01 14 19 5E 28 0E 08 0F
0000:0064 F8 17 5E 48 04 01 14 E8 07 18 D2 81 02 7F

```

首先，以下内容为 AJE 文件头信息：

```
61 CC 1C 41 4C 5F 5F 5F 31 2E 30 F2 62 20
```

其包含如下四部分的内容：

CRC	版本信息	是否压缩	内存大小
61 CC 1C	41 4C 5F 5F 5F 31 2E 30	F2	62 20

其次是厂商信息：“56 01”。

再次，从第一个“5E”开始为 SVF 数据内容。其中“5E”为行头标记，其后紧接着该行数据的长度。之后就是 SVF 命令编码及数据。SVF 命令与 AJE 编码的对应关系如下表所示：

SVF 命令	AJE 编码
TRST OFF;	01 03
ENDIR IDLE;	05 01
ENDDR IDLE;	06 01
STATE RESET;	04 00
STATE IDLE;	04 01
FREQUENCY 1E6 HZ;	07 C0 84 3D
SIR 8 TDI (06) SMASK (FF);	0E 08 0F 60 17
RUNTEST 15 TCK;	04 01 14 19
SDR 32 TDI (00000000) SMASK (FFFFFFFF) TDO 03004C37) MASK (FFFFFFFF);	0D 20 0F 00 00 00 00 10 EC 32 00 C0 11 FF FF FF FF 17
RUNTEST 15 CLK;	04 01 14 19
SIR 8 TDI (01) SMASK (FF);	0E 08 0F 80 17
RUNTEST 15 CLK;	04 01 14 19
SIR 8 TDI (1F);	0E 08 0F F8 17
RUNTEST 200000 TCK;	04 01 14 E8 07 18 D2 81 02

最后，“7F”为文件结束符。

4.4 SVF 命令支持

AJE 格式支持的 SVF 命令如下表所示：

名称	编码	描述
TRST	0x01	Jtag reset pin 行为
STATE	0x04	移动至下一个状态
ENDIR	0x05	SIR 之后的 jtag tap 状态
ENDDR	0x06	SDR 之后的 jtag tap 状态
FREQUENCY	0x07	JTAG 状态机时钟频率
TDR	0x09	尾部级联
TIR	0x0A	尾部级联
HDR	0x0B	头部级联
HIR	0x0C	头部级联
SDR	0x0D	进入 Shift dr 并输入数据
SIR	0x0E	进入 shift ir 并输入数据
TDI	0x0F	TDI 的输入值
TDO	0x10	TDO 的输入值
MASK	0x11	数据比较标记
RUNTEST	0x13	Jtag idle 状态保持时间
TCK	0x14	TCK 时钟
RSDR	0x1C	扩展指令，RSDR 用法和 SDR 类似，它将执行 2 次 SDR 指令

4.5 JTAG 操作编码

AJE 支持的 JTAG 操作编码如下表所示：

名称	编码	描述
RESET	0x00	Jtag tap reset 状态
IDLE	0x01	Jtag tap Idle 状态
SHIFTIR	0x02	Jtag tap shift ir 状态
SHIFTDR	0x03	Jtag tap shift dr 状态
DRPAUSE	0x04	Jtag tap dr pause 状态
IRPAUSE	0x05	Jtag tap ir pause 状态

5 SVF2AJE 转换工具使用

使用 TD 软件 device_chain 工具将 bit 文件可直接转换成 aje 文件；也可使用 device_chain 工具产生 svf 文件，供 svf2aje 工具使用，手动转换输出 aje 文件。

在 cmd 窗口，切换到 svf2aje 工具路径下，如图 1 所示：

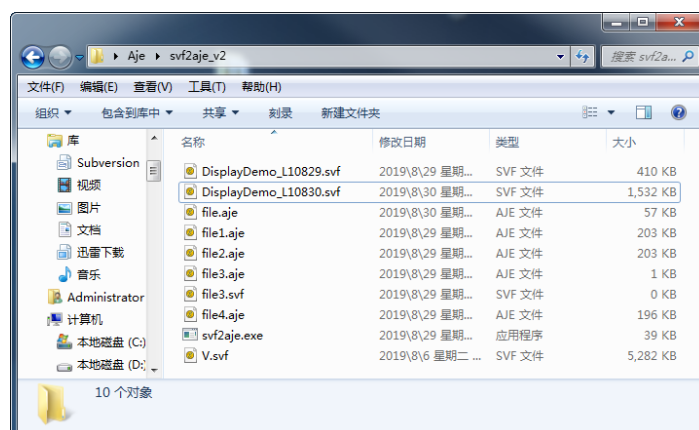


图 1 AJE 工具路径

svf2aje 使用帮助如下

```
C:\Users\Administrator\Desktop\Aje\exe>svf2aje.exe
```

```
Usage: svf2aje [ -help |  
               [ -svffile <svf file path> [ -ajefile <aje file path> ] ] ]
```

Descriptions:

```
-help:    Displays usage.  
-svffile: Specifies the input SVF file.  
-ajefile: Specifies the input AJE file.
```

Examples:

```
svf2aje -svffile c:\file.svf -ajefile c:\file.aje
```

默认产生的.aje 文件为压缩版本，命令如下：

```
svf2aje.exe -svffile V.svf -ajefile file.aje
```

若需产生的非压缩.aje 文件版本，命令如下：

```
svf2aje.exe -svffile V.svf -ajefile file.aje -full
```

工具所在目录下，会保存用户产生的*.aje 文件，工具运行无误，如下图 2 所示：

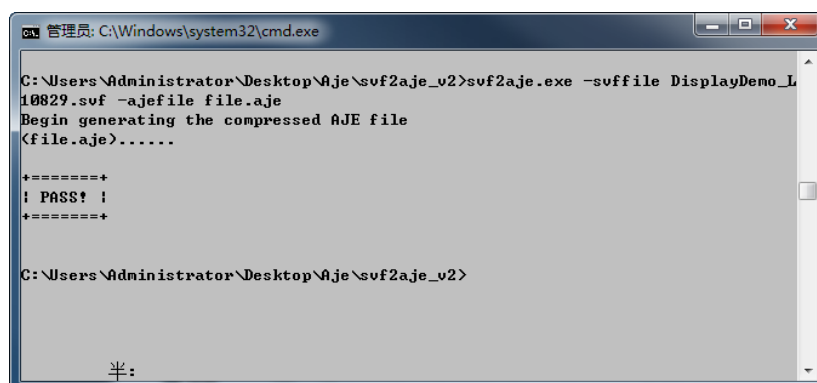


图 2 svf2aje 产生 AJE 文件

如果将.aje 还原成.svf 文件，可通过下述指令，进行转换，会保存用户产生的*.svf 文件，工具运行无误，如下图 3 所示：

```
svf2aje.exe -ajefile file.aje -svf file file4.svf -r
```

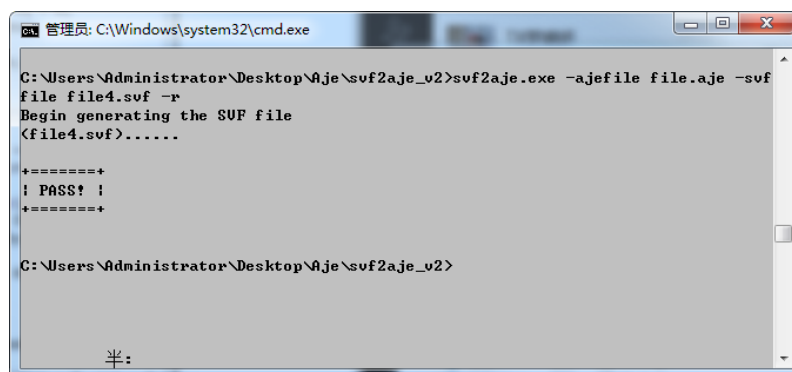


图 3 svf2aje 产生 SVF 文件

在 Linux 平台上，可运行 svf2aje Linux 版本，命令及其使用如图 4 所示。

```
[#11#11:30:33 [redacted]@ubuntu ~/Aje]$./svf2aje
Usage: svf2aje [ -help |
               [ -svffile <svf file path> [ -ajefile <aje file path> ] ]
               ]
Descriptions:
  -help:      Displays usage.
  -svffile:   Specifies the input SVF file.
  -ajefile:   Specifies the output AJE file.
Examples:
  svf2aje -svffile c:\file.svf -ajefile c:\file.aje

[#12#11:30:39 [redacted]@ubuntu ~/Aje]$./svf2aje -svffile ef3l90.svf -ajefile ./Test.aje
Begin generating the compressed AJE file
(./Test.aje).....

+=====+
| PASS! |
+=====+

[#13#11:30:42 FPGADeveloper@ubuntu ~/Aje]$ls
ef3l90.aje  ef3l90_linux.aje  ef3l90.svf  svf2aje  Test.aje
[#14#13:36:55 [redacted]@ubuntu ~/Aje]$
```

图 4 svf2aje Linux 版本

对于新版本的 svf2aje 转换工具，在支持 SVF for SPI 同时，也新增对 SVF for SRAM 配置的支持。

SRAM 配置，可通过下述命令对指令长度进行限定以防止 SRAM 配置文件过长而导致的：

```
svf2aje.exe -svffile PH4Led_L1SRAM.svf -ajefile PH4LedSRAM.aje -max_size 64
```

其中 max_size 值为 1、2、4、8、16、32、64，默认转换为 64 (max_size*1000)。

TD 新版本支持，在生成 SVF 文件时，由原操作固定等待时长，更改为轮询机制，可通过 TD Console 生成带 loop 指令的 SVF 文件。

```
svfgen -scan_mode prog_spi -level 2 -bit Led_Core_Runs/phy_1/Led_Core_compress.bit -svf
al_devicechain/Led_Core_L2flash.svf -opt no_comment -tool tde -bg_mode no -len_mode
normal -rf_mode yes -bypass 11111111_11111111 -flashsize 128 -freq 1MHz -enable_loop
```

6 AJE2JTAG 应用层代码移植

6.1 系统文件

AJE2JTAG 应用层代码包含了六个文件(模块)，描述如下：

main.c: 主函数入口；

aje2vec.c: AJE 文件转换成 vector；

vec.c: Vector 实现;

opcode.h: SVF 操作指令;

decode.c: 读取.aje 文件;

lzw_lib.c: LZW 解压缩算法;

ajeutil.c: aje 用到的其他转换函数文件;

gpio.c: GPIO 底层驱动, JTAG 功能管脚映射;

6.2 代码移植

上述代码中, 主要涉及到 gpio.c 及其 gpio.h 的修改。

gpio.h 部分文件如下, 对 GPIO 基地址及其指针进行定义声明。

```
#define JTAG_TMS      0x41200000
#define JTAG_TCK      0x41210000
#define JTAG_TDI      0x41220000
#define JTAG_TDO      0x41230000
#define GPIO_MAP_SIZE 0x1000    // Page size is 4Kb

volatile u32 *mapped_jtag_tms ;// Direction->Output
volatile u32 *mapped_jtag_tck ;// Direction->Output
volatile u32 *mapped_jtag_tdi ;// Direction->Output
volatile u32 *mapped_jtag_tdo ;// Direction->Input

volatile u32 *mapped_base;
```

gpio.c 部分文件如下, 对 GPIO 初始化及其读写函数定义, 部分代码如下。

```
// Set GPIO Direction
void GPIO_SetDir(volatile u32 *Gpio_Baseaddr, volatile u32 Value)
{
    *(Gpio_Baseaddr + GPIO_TRI) = Value;
}

//TMS Write function
void TMS_Wr(u8 value)
{
    *(mapped_jtag_tms + GPIO_DATA) = value ? 1: 0;
}
//TCK Write function
void TCK_Wr(u8 value)
{
    *(mapped_jtag_tck + GPIO_DATA) = value ? 1: 0;
}
//TDI Write function
void TDI_Wr(u8 value)
{
    *(mapped_jtag_tdi + GPIO_DATA) = value ? 1: 0;
}
//TDO Read function
u8 TDO_Rd(void)
{
    u8 ReadValue = 0;
    ReadValue = *(mapped_jtag_tdo + GPIO_DATA);
    return (ReadValue & 0x1);
}
```

上述驱动, 可根据平台特性, 进行自定义移植开发。

6.3 调试与错误代码

调试阶段，可以在 `opcode.h` 开启 `debug`。

```
#define AJE DEBUG
```

配置阶段会打印相关信息，供调试使用，如图 5 所示。

[illegible]

图 5 打印调试信息

在调试过程中可能会出现 **Failed** 的情况，可根据下表指示代码进行修改。

AJE_FILE_OPEN_FAIL	-6	Value returned when the AJE file cannot be open
AJE_FILE_INVALID	-7	Value returned when the AJE file is invalid
AJE_INVALID_COMMAND	-10	Value returned when command is invalid
AJE_TRANSFER_FAIL	-16	Value returned when data transferred failed
AJE_VERIFY_FAIL	-17	Value returned when verify failed

6.4 测试验证

在 ARM 平台上，将 4 个 GPIO 连接到 FPGA 的 JTAG 接口上，AJE2JTAG 硬件平台搭建如下，如图 6 所示：

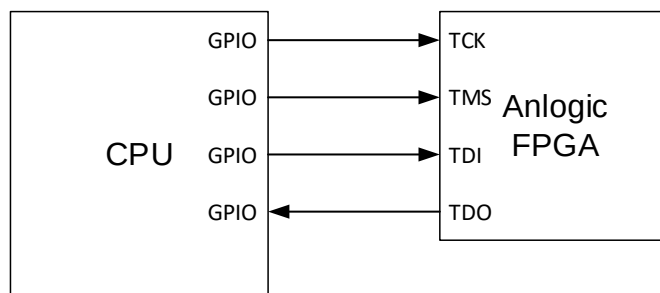


图 6 AJE2JTAG 验证平台

将 aje 文件嵌入到工程中，main 函数代码如下，AJE2JTAG 函数为 AjeToVec()。

```

int main(int argc, char *argv[])
{
    int rtn_code;

    char * aje_file = "/file.aje";

    Cfg_UIO();

    rtn_code = Anlogic_AjeToVec(aje_file);

    if(rtn_code == AJE_OK) {
        printf(" Success!\n\n");
    } else {
        printf(" Fail!\n");
        printf(" Error Code = % d!\n\n",rtn_code);
    }

    return rtn_code;
}
  
```

在 gpio.c 中，定义 TMS/TCK/TDI/TDO 相关读写函数，此 Demo 通过 UIO 方式对 CPU 外设 GPIO 进行操作，部分代码如下。

```

...
    fd = open("/dev/uio0", O_RDWR);
    mapped_base = mmap(0, GPIO_MAP_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED,
fd, 0);
    if (mapped_base == -1)
        printf("\n Error opening JTAG_TMS device file \n");
    else
        printf("\n Opened device JTAG_TMS file successfully %d\n",fd);
    mapped_jtag_tms = mapped_base ;
    printf("\n TMS:\n Memory mapped at address %p.\n", mapped_jtag_tms);

    fd = open("/dev/uio1", O_RDWR);
    mapped_base = mmap(0, GPIO_MAP_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED,
fd, 0);
    if (mapped_base == -1)
        printf("\n Error opening JTAG_TCK device file \n");
    else
        printf("\n Opened device JTAG_TCK file successfully %d\n",fd);
    mapped_jtag_tck = mapped_base ;
    printf("\n TCK:\n Memory mapped at address %p.\n", mapped_jtag_tck);
...
  
```

在 Linux 系统中，待配置文件 file.aje，AJE2JTAG 程序 Aje_Compress.elf，运行结果如图 7 所示。

```
[root@20190731]#./Aje_Compress.elf
opened device JTAG_TMS file successfully 3
TMS:
Memory mapped at address 0x36fa2000.
opened device JTAG_TCK file successfully 4
TCK:
Memory mapped at address 0x36fa0000.
opened device JTAG_TDI file successfully 5
TDI:
Memory mapped at address 0x36f9f000.
opened device JTAG_TDO file successfully 6
TDO:
Memory mapped at address 0x36e21000.
JTAG GPIO Direction TMS: -> 0x0000 output
JTAG GPIO Direction TCK: -> 0x0000 output
JTAG GPIO Direction TDI: -> 0x0000 output
JTAG GPIO Direction TDO: -> 0xFFFF Input
Success!
[root@20190731]#
```

图 7 运行结果

7 AJE2JTAG 内核层代码移植

7.1 系统文件

AJE2JTAG 内核层代码包含了六个文件(模块)，描述如下：

aje2vec.c: AJE 文件转换成 vector;

vec.c: Vector 实现;

opcode.h: SVF 操作指令;

decode.c: 读取.aje 文件;

lzw_lib.c: LZW 解压缩算法;

ajeutil.c: aje 用到的其他转换函数文件;

jtag_gpio.c: GPIO 管脚，JTAG 功能管脚映射;

7.2 代码移植

上述代码中，主要涉及到 jtag_gpio.c 及其 jtag_gpio.h 的修改。

GPIO 驱动详见 jtag_gpio.c，通过 ioremap，直接操作物理地址+偏移地址实现。用户可将 cfg_jtag_gpio()，替换成相应的自定义函数即可。

```
/* main function*/
```

```

int Anlogic_AjeToVec(const char* aje_file, char* message) {
    int rtn_code = AJE_OK;

    if (aje_file == NULL) {
        return AJE_FILE_EMPTY;
    }

    cfg_jtag_gpio();

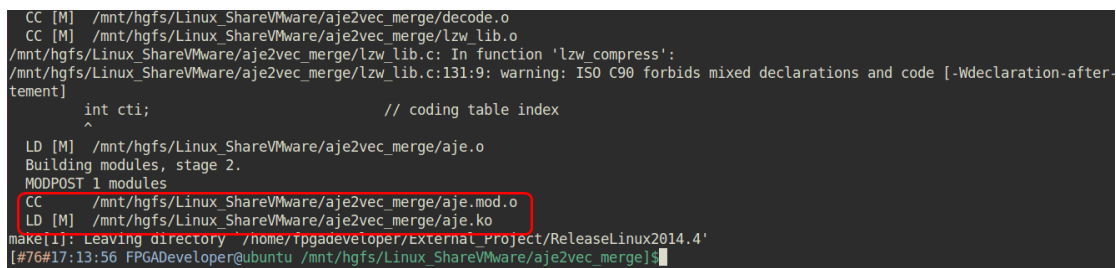
    Anlogic_Init();

    /* read aje file content and save */
    rtn_code = Anlogic_ReadAjeContent(aje_file, message);

    if (rtn_code != AJE_OK) {
        return rtn_code;
    }
}

```

编译正确后，将产生 aje.ko。



```

CC [M] /mnt/hgfs/Linux_ShareVMware/aje2vec_merge/decode.o
CC [M] /mnt/hgfs/Linux_ShareVMware/aje2vec_merge/lzw_lib.o
/mnt/hgfs/Linux_ShareVMware/aje2vec_merge/lzw_lib.c: In function 'lzw_compress':
/mnt/hgfs/Linux_ShareVMware/aje2vec_merge/lzw_lib.c:131:9: warning: ISO C90 forbids mixed declarations and code [-Wdeclaration-after-
temment]
    int cti;                                // coding table index
    ^
LD [M] /mnt/hgfs/Linux_ShareVMware/aje2vec_merge/aje.o
Building modules, stage 2.
MODPOST 1 modules
CC [M] /mnt/hgfs/Linux_ShareVMware/aje2vec_merge/aje.mod.o
LD [M] /mnt/hgfs/Linux_ShareVMware/aje2vec_merge/aje.ko
make[1]: Leaving directory '/home/tpgadeveloper/external_project/ReleaseLinux2014.4'
[#76#17:13:56 FPGADeveloper@ubuntu /mnt/hgfs/Linux_ShareVMware/aje2vec_merge]$

```

图 8 驱动编译

ARM 硬件平台上，调用方法如下：

```
insmod aje.ko aje_file="1.aje"
```

其中 1.aje 为 *.svf 转成的 *.aje 文件。

8 延时实现与校准

对于延时部分，受硬件平台参数影响，在实际运行中，可能需要对 RUNTEST 部分进行校准(如若平台出来速度过快，RUNTEST 等待时间不够，可能会导致配置失败)，在 vec.c 中，默认提供的 RUNTEST 部分实现的函数如下，在 Linux 应用层，可通过 usleep 的方式实现延时（TD 产生 SVF 文件，TCK 默认频率为 1MHz，即单位为 us）。

```

void Anlogic_ProcessRunTestTck (int num) {

    TDI_Wr(1);
    TMS_Wr(1);
    TCK_Wr(1);
}

```

```
        usleep(num);
    }

    对于 SVF 文件参考频率若为其他频率，可以根据 RUNTEST 值进行换算。在 Uboot 环境下，可通过调用 udelay 函数实现。对于其他环境，可通过 for 循环进行实现，实现代码如下，对于延时的准确性，可通过调整 num 进行设定。
```

```
void Anlogic_ProcessRunTestTck (int num) {
    int i = 0;
    TDI_Wr(0);
    TMS_Wr(0);
    for(i = 0; i < num; ++i) {
        TCK_Wr(1);
        TCK_Wr(0);
    }
}
```

在主函数中，调用 Anlogic_Calibration()，TCK 管脚输出脉冲，脉冲间隔应在 1ms（当 SVF 参考频率为 1M 时，若 TCK 脉冲间隔至少 1ms，那么与 SVF 文件中的 RUNTEST 时延相符），波形如图 9 所示。

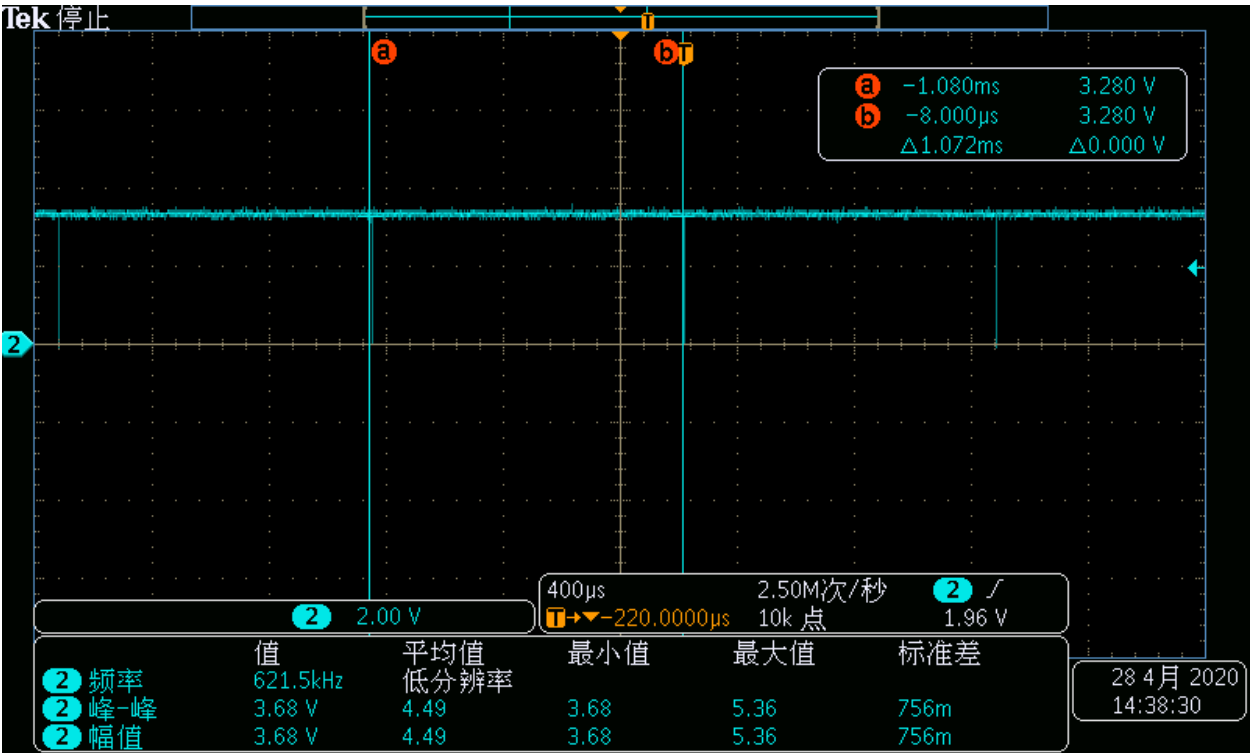


图 9 TCK 脉冲间隔

9 AJE 与 TDE 移植对比及其注意事项

Aje 与 Tde 版本配置 EF2/EF3/EG4 器件所耗时长如下，单位：秒（测试平台：CPU:667MHz/DDR3:533MHz）。

版本	EF2L45	EF3L90	EG4X20
Aje 内核态版本	14. 84	28. 21	58. 07

Aje 应用态版（单任务）	6. 48	12. 25	25. 31
Tde 版本	10. 04	18. 89	39. 14

Aje (仅应用层) 与 Tde 移植对比如下，

版本	源代码	移植	配置文件
AJE 应用层 版本	√	提供《TN008_Aje 移植指南》，移植过程中，主要对 <code>gpio.c</code> 中的代码进行修改，结合客户硬件平台，对 TCK/TMS/TDI/TDO 进行绑定，提供校准函数；	TD 产生*.aje 文件。
TDE 版本	√	提供《TN006_Tde 移植指南》，移植过程中，主要对 <code>hardware.c</code> 中的代码进行修改，结合客户硬件平台，对 TCK/TMS/TDI/TDO 进行绑定，提供校准函数；	TD 产生*.tde 文件。

不论是 Aje 移植还是 Tde 移植，在移植过程中都需要对 2 个 Tck 脉冲间的时长进行校准。移植可按照下述 3 个步骤：

- Aje/Tde 代码移植，对 TMS/TCK/TDI/TDO 驱动（底层）部分分配；
- 移植后，需要对 2 个 Tck 脉冲间进行校准；
- 下载配置测试；

10 TIR/TDR/HIR/HDR 支持

对于后续发布的 TD 版本，在 Device chain 界面上，对 SVF 文件的产生，增加 TIR/TDR/HIR/HDR 指令使能选项。从应用层 V6.5.222923(内核层 V6.5.24288)开始，对于级联器件，既支持相对位置的 SVF 文件(TIR/TDR/HIR/HDR 使能)，又支持绝对位置的 SVF 文件(TIR/TDR/HIR/HDR 不使能)。

AJE 版本	TIR/TDR/HIR/HDR 不使能	TIR/TDR/HIR/HDR 使能
~V6.5.19512(应用层)	支持	-
V6.5.222923(应用层)~	支持	支持
~V6.4(应用层)	支持	-
V6.5.24288(内核层)~	支持	支持

11 读取 Flash 数据

在配置完成后，用户可根据需求再次通过 JTAG 再次对 Flash（内置 Flash）进行用户数据读或写，

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f	
00000000h:	FF	DD	66	BB	22	FF	FF	FF	A2	FF	FF	FF	FF	FF	FF	FF	; 峯? ?
00000010h:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	;
00000020h:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	;
00000030h:	CC	55	AA	33	F0	00	00	06	00	00	8C	3B	2B	82	C2	00	; 藕??...?+俚.
00000040h:	00	06	01	00	15	00	49	B4	C3	00	00	06	D0	30	4B	B0	;I疵...?K?
00000050h:	C9	F0	C7	00	00	06	04	16	01	1E	A4	D5	C8	00	00	06	; 绅?.....ふ?...?
00000060h:	00	00	04	80	98	88	C1	00	00	06	00	0D	1E	0A	D7	65	; ...€模?.....譏
00000070h:	CA	00	00	06	00	00	00	00	A3	84	C4	00	00	06	00	30	; ?..... ?...0
00000080h:	0E	30	E4	C3	F1	00	00	04	00	00	78	AF	EC	F0	04	16	; .0涿?...x ?.
00000090h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000a0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000b0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000c0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000d0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000e0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000000f0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000100h:	12	34	56	78	90	00	11	22	33	44	55	88	99	aa	CC	DD	; .4Vx?."3DU达 ?
00000110h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000120h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000130h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000140h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000150h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000160h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000170h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000180h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000190h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000001a0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	43 8B	;C?
000001b0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000001c0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000001d0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000001e0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
000001f0h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;
00000200h:	34	56	78	9A	DF	EC	DE	EE	EE	EE	EE	EE	FF	FF	FF	F0	; 4Vx氮烫铭铭?
00000210h:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	;

图 10 更改 bin 文件

通过 TD Device chain 将 bin 文件进行转换成 SVF 文件，截取部分 SVF 指令；同时可生成回读校验的区域，将 MASK 更改成 RMASK 的 SVF 指令，如图 11 所示。

```

00000000000000000000000000000000c0a5) RMASK (1fffffffffffffffffffffffffffffffffffffffff
000000000000000000000000000008000c0a5) RMASK (1fffffffffffffffffffffffffffffffffffffffff
000000000000000000000000000004000c0a5) RMASK (1fffffffffffffffffffffffffffffffffffffffff
00000000000000000000000000000c000c0a5) RMASK (1fffffffffffffffffffffffffffffffffffffffff
00000000000000000000000000002000c0a5) RMASK (1fffffffffffffffffffffffffffffffffffffffff

```

图 11 增加 RMASK 指令

运行 AJE 加载程序，通过调试功能将回读输出的值打印到终端，如图 12 所示，回读输出数据与原始数据一致。

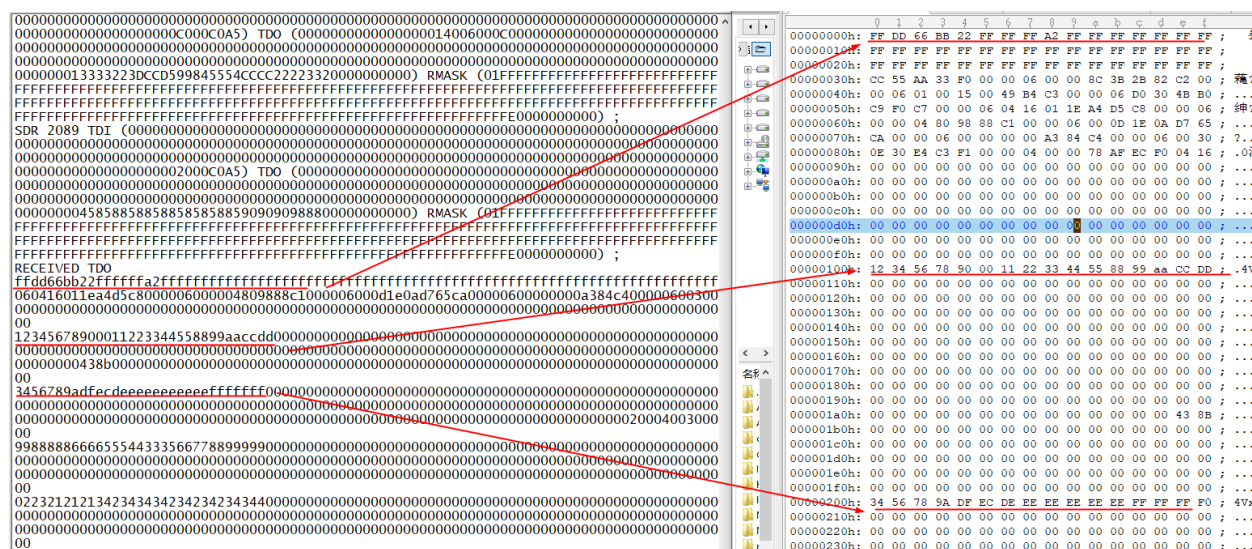


图 12 回读打印输出对比

对于数据的回读，可调用 `Anlogic PrintRdbkValue()`函数获取。

回读指令的构造，可参考下述例程，以 EF3 器件为例：

SDR 2092 TDI

[illegible]

RMASK

[illegible]

SDR

2092

TDI

[illegible]

SDR

2092

TDI

[illegible]

[illegible]

编程操作的同步码固定为 0xa5:

编程操作的操作码固定为 0x40:

编程的地址为 3 字节，低 bit 位在前。JTAG 指令里的地址和实际地址的对应关系如下：

- a) 000000, 对应 Flash 的 page 地址为 0x000000;
- b) 008000, 对应 Flash 的 page 地址为 0x000100;
- c) 004000, 对应 Flash 的 page 地址为 0x000200;

对于读操作，可根据需求，将读 Flash 读 page 地址根据上述对应规则进行转换，更新即可。若需对其转换规则和格式作更详尽的了解，可参考 EF2/EF3 等器件对应的第三方烧录工具指南。

版本信息

日期	版本	说明
2019/08/06	1.0	初版建立
2019/08/30	1.1	增加压缩功能
2019/12/11	1.2	增加 svf2aje linux 版本
2020/03/24	1.3	增加延时校准方法
2020/03/27	1.4	增加 7 AJE2JTAG 内核层代码移植
2020/04/28	1.5	对应用态代码增加 Anlogic_Calibration 函数，进行延时校准
2020/05/14	1.6	增加 9 AJE 与 TDE 移植对比及其注意事项
2020/10/10	1.7	在 5 SVF2AJE 转换工具使用，增加对 SVF for SRAM 配置的支持说明
2020/11/04	1.8	增加 TIR/TDR/HIR/HDR 指令支持的说明
2020/12/14	1.9	修改因版本更新对应的函数名 增加读取 Flash 数据章节
2020/12/16	2.0	详细描述回读指令的构造
2021/09/09	2.1	增加 SRAM 转换指定长度功能 增加生成 loop 功能的操作指令

版权所有© 2021 上海安路信息科技股份有限公司

未经本公司书面许可，任何单位和个人不得擅自摘抄、复制、翻译本文档内容的部分或全部，并不得以任何形式传播。

免责声明

本文档并未授予任何知识产权的许可，并未以明示或暗示，或以禁止发言或其它方式授予任何知识产权许可。除安路科技在其产品的销售条款和条件中声明的责任之外，安路科技概不承担任何法律或非法律责任。安路科技对安路科技产品的销售和/或使用不作任何明示或暗示的担保，包括对产品的特定用途适用性、适销性或对任何专利权、版权或其它知识产权的侵权责任等，均不作担保。安路科技对文档中包含的文字、图片及其它内容的准确性和完整性不承担任何法律或非法律责任，安路科技保留修改文档中任何内容的权利，恕不另行通知。安路科技不承诺对这些文档进行适时的更新。